JointShocks R Package

Johnson Oliyide and Thomas Wiesen

Contents

1 Purpose **1**

2 Installation Procedure **2**
2.1 Loading your own data 2
2.2 Sample data used in this vignette 2

3 Empirical Analysis **3**
3.1 Wiesen and Beaumont (2024) -The Joint Impulse Response Function and Joint Forecast Error Variance Decomposition 3
3.2 Lastrapes and Wiesen (2021) and Lastrapes and Wiesen (2025) - Joint Spillover Metrics . . . 14

1 Purpose

Economic interdependence and financial globalization have intensified the need for accurate measures of shock transmission across macroeconomic and financial systems. To this end, the **JointShocks** R package provides researchers and practitioners with robust tools to estimate the joint spillover metrics proposed in Lastrapes and Wiesen (2021) and Lastrapes and Wiesen (2025).

Additionally, it offers functionalities to compute the joint impulse response function and the joint forecast error variance decomposition introduced in Wiesen and Beaumont (2024).

For comparative purposes, the package also includes:

- The **generalized spillover index** and metrics of Diebold and Yilmaz (2012) and Diebold and Yılmaz (2014), and
- The **generalized impulse response function** and **generalized forecast error variance decomposition** of Koop, Pesaran, and Potter (1996) and Pesaran and Shin (1998).

The goal of this package is to streamline the computation of these metrics-across both **full-sample** and **rolling window** applications-within a user-friendly and reproducible framework. It bridges theoretical contributions from the academic literature with accessible implementation tools for empirical work in macro-finance, banking, and monetary economics.

The remainder of this documentation proceeds as follows: Section 2 briefly discusses the installation procedure. Section 3.1 provides and discusses the replication of key results from Wiesen and Beaumont (2024), including the estimation of their Joint Impulse Response Functions (jIRFs). This involves implementing their bootstrap procedures, as well as computing the Generalized Impulse Response Functions (gIRFs) and the Generalized Forecast Error Variance Decomposition (gFEVD). Section 3.2 focuses on estimating the joint spillover metrics as developed in Lastrapes and Wiesen (2021) Lastrapes and Wiesen (2025) and the generalized spillover metrics in Diebold and Yilmaz (2012) and Diebold and Yılmaz (2014).

```
knitr::opts_chunk$set(echo = TRUE)
options(width = 80)
```

```
# Wrap long lines of printed output so they don't run off the edge of the
# page in the PDF version of this vignette.
hook_output <- knitr::knit_hooks$get("output")
knitr::knit_hooks$set(output = function(x, options) {
  if (!is.null(options$linewidth)) {
    x <- knitr:::split_lines(x)
    x <- ifelse(nchar(x) > options$linewidth,
                strwrap(x, width = options$linewidth), x)
    x <- paste(x, collapse = "\n")
  }
  hook_output(x, options)
})
knitr::opts_chunk$set(linewidth = 80)
#install.packages("xfun")
```

2 Installation Procedure

To begin using the **JointShocks** package, install it from GitHub using the `devtools` package (only needed once):

```
#install.packages("devtools")
devtools::install_github("JohnsonOliyide/JointShocks")
```

Then load it as follows:

```
library(JointShocks)
```

Because the package relies on vector autoregressions, be sure to load the `vars` package:

```
library("vars")
```

2.1 Loading your own data

The package currently supports data files in `.csv` or `.xlsx` format. The first column must represent dates. To read `.xlsx` files, load the `readxl` package:

```
library("readxl")
```

To load your own dataset, use the `read_data()` function, pointing it at the path to your file and specifying the format of the dates in your file's first column. For example, if you had a file called `"my_data.csv"` with dates formatted like `1/5/2017`, you would run:

```
Data_date <- read_data("my_data.csv", date_format = "%m/%d/%Y")

Data <- Data_date$Data
date <- Data_date$date
```

This returns a list with `Data` (a matrix of your variables) and `date` (a vector of the corresponding dates), ready to be used with `vars::VAR()` and the rest of the functions in this package.

2.2 Sample data used in this vignette

For replication, the remainder of this vignette uses datasets bundled with the package from Wiesen and Beaumont (2024), which contain the volatility of eleven major banks from three different periods: pre-pandemic (1/5/2017-1/31/2020), pandemic (2/3/2020-2/3/2021), and full sample (1/5/2017-2/3/2021). The banks used are Bank of America (BAC), Bank of New York Mellon (BK), Barclays (BCS), Citigroup (C),

Credit Suisse (CS), Deutsche Bank (DB), Goldman Sachs (GS), JP Morgan Chase (JPM), Morgan Stanley (MS), State Street (STT), and Wells Fargo (WFC).

Load the pre-pandemic sample data as follows:

```
data(Volatility_data_pre)

Data <- Volatility_data_pre
date <- attr(Volatility_data_pre, "date")
```

3 Empirical Analysis

3.1 Wiesen and Beaumont (2024) -The Joint Impulse Response Function and Joint Forecast Error Variance Decomposition

We refer users to Wiesen and Beaumont (2024) for a complete explanation of the methodology and its empirical contributions.

Estimate a VAR model

```
#------------- Run a VAR model------
# Estimate VAR with 1 lag, as used in the paper
VAR_model <- VAR(Data, p = 1, type = "const")
```

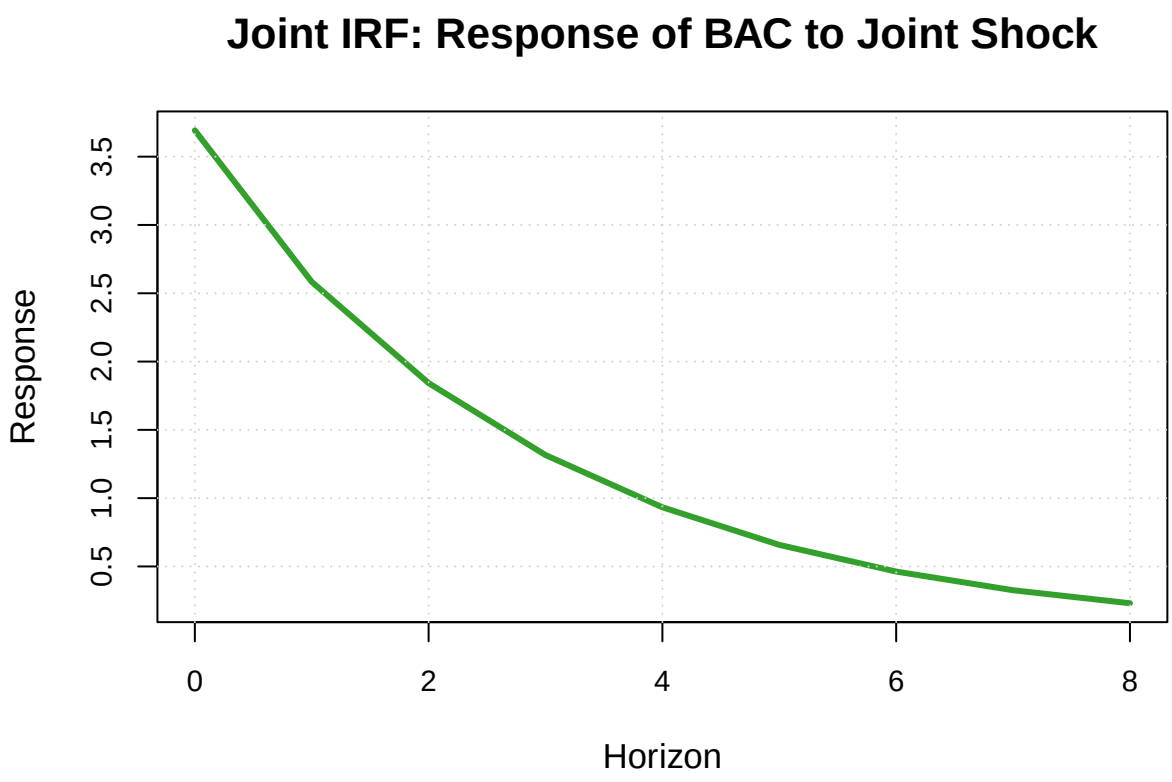
3.1.1 Joint Impulse Response Function (jIRF)

To compute the joint impulse response function involving shocks from Barclays (BCS), Credit Suisse (CS), and Deutsche Bank (DB), run this:

```
jIRF <- run_full_jIRF(VAR_model, from_set = c("BCS", "CS", "DB"), H = 8)
```

You can plot the jIRF response of Bank of America. The result it produces shows how the equity market of Bank of America respond over the next eight days to a joint, simultaneous one standard deviation shocks from Barclays, Credit Suisse and Deutsche Banks. Since `jIRF` is a labeled matrix, we can plot it directly using base R:

```
# Plot jIRF for specific response
horizon <- 0:8
plot(horizon, jIRF["BAC", ], type = "l", lwd = 3, col = "#33a02c",
     xlab = "Horizon", ylab = "Response",
     main = "Joint IRF: Response of BAC to Joint Shock",
     cex.main = 1.3, cex.lab = 1.1, cex.axis = 0.95)
abline(h = 0, col = "gray40", lty = 2)
grid()
```



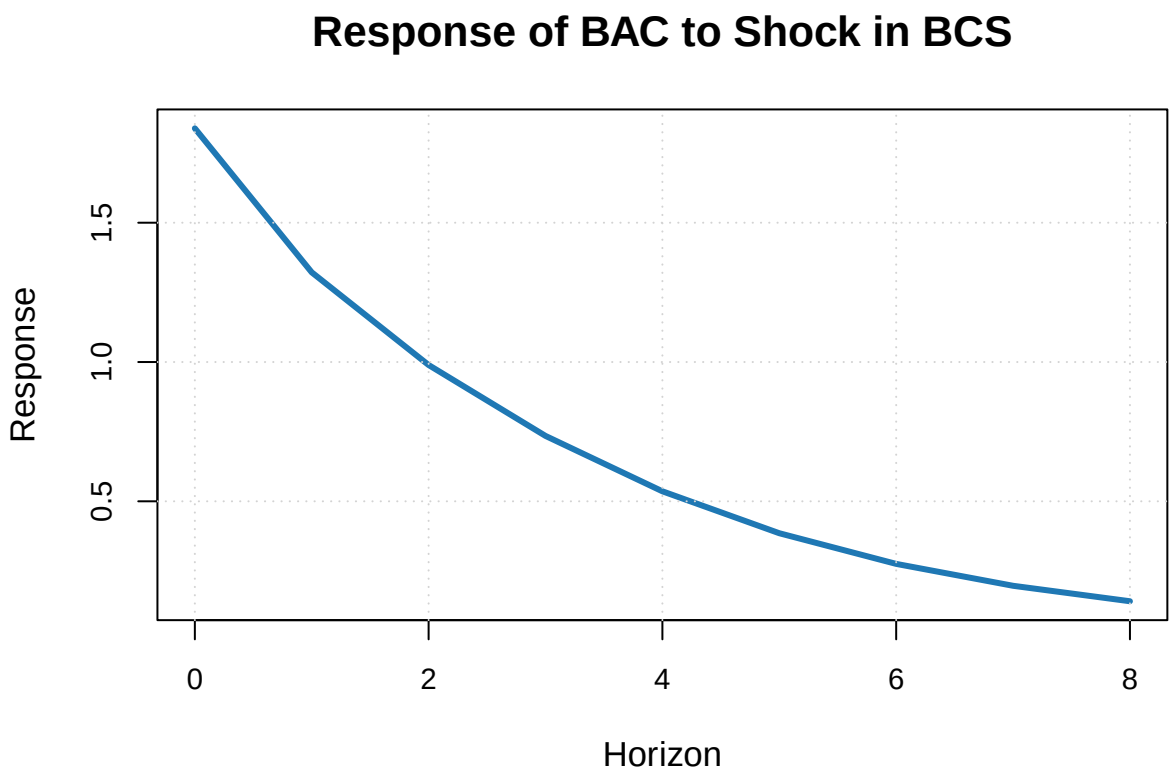
If you're interested in the gIRF of Pesaran and Shin (1998) without confidence bounds, run:

```
gIRF <- run_full_gIRF(VAR_model, H = 8) # Run the gIRF
```

This returns the entire set of gIRF results, showing how each variable responds to a one-standard-deviation shock in every other variable.

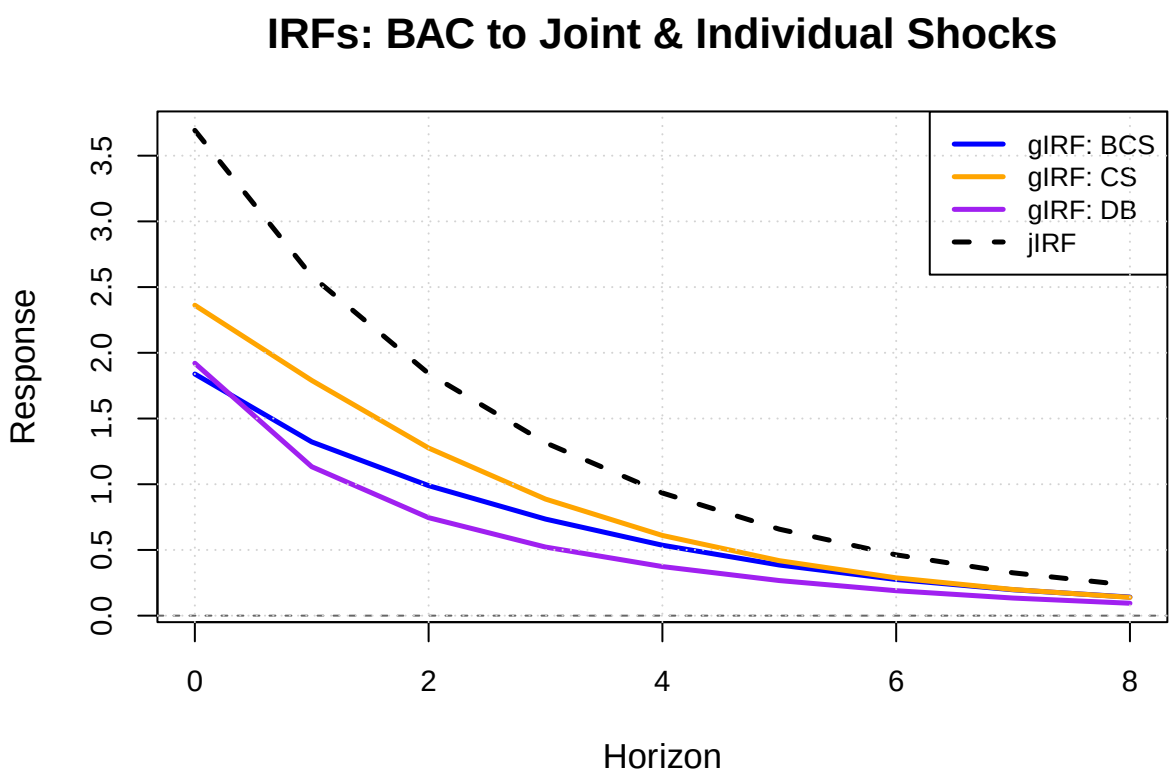
To visualize a specific response-shock pair — for instance, the effect of a shock from Barclays on Bank of America:

```
plot(horizon, gIRF["BAC", "BCS", ], type = "l", lwd = 3, col = "#1f78b4",
     xlab = "Horizon", ylab = "Response",
     main = "Response of BAC to Shock in BCS",
     cex.main = 1.3, cex.lab = 1.1, cex.axis = 0.95)
abline(h = 0, col = "gray40", lty = 2)
grid()
```



You may compare the jIRF against individual gIRFs using:

```
# Combined IRFs as in jIRF paper
from_set <- c("BCS", "CS", "DB")
g_colors <- c("blue", "orange", "purple")
j_color <- "black"
g_vals_list <- lapply(from_set, function(s) gIRF["BAC", s, ])
y_range <- range(c(unlist(g_vals_list), jIRF["BAC", ]))
plot(horizon, jIRF["BAC", ], type = "n",
     ylim = y_range, xlab = "Horizon", ylab = "Response",
     main = "IRFs: BAC to Joint & Individual Shocks",
     cex.main = 1.3, cex.lab = 1.1, cex.axis = 0.95)
for (i in seq_along(from_set)) {
  lines(horizon, g_vals_list[[i]], col = g_colors[i], lwd = 2.5, lty = 1)
}
lines(horizon, jIRF["BAC", ], col = j_color, lwd = 2.5, lty = 2)
abline(h = 0, col = "gray40", lty = 2)
legend("topright", legend = c(paste0("gIRF: ", from_set), "jIRF"),
       col = c(g_colors, j_color), lty = c(rep(1, length(from_set)), 2), lwd = 2.5, cex = 0.85)
grid()
```



Note: To explore a different set of shock contributors, rerun `run_full_jIRF()` with a new `from_set` and recompute `jIRF`/`gIRF` before re-plotting.

3.1.2 Joint Forecast Error Variance Decomposition (jFEVD)

Users of this package can also calculate the joint forecast error variance decomposition (jFEVD) of Wiesen and Beaumont (2024). This jFEVD measures the proportion of a variable's forecast error variance that can be jointly explained by a specified set of multiple shocks.

We recommend referring to Wiesen and Beaumont (2024) for a detailed explanation of the model's theoretical advantages and empirical applications.

To illustrate, the jFEVD result can be interpreted as follows: approximately **16%** of the H=30 step forecast error variance of **Bank of America (BAC)** is attributable to joint shocks from **Barclays (BCS)**, **Credit Suisse (CS)**, and **Deutsche Bank (DB)**.

```
#-------------Joint FEVD------
# Run and present jFEVD result
jfevd <- run_full_jFEVD(VAR_model, from_set = c("BCS", "CS", "DB"), H = 30)
jfevd
```

```
##         BAC         BK        BCS          C         CS         DB         GS
## 0.15952241 0.05223781 0.96668595 0.16460344 0.95693593 0.94891627 0.14332551
##         JPM         MS        STT        WFC
## 0.16926043 0.13025529 0.05798865 0.08725354
```

3.1.3 Generalized Forecast Error Variance Decomposition (gFEVD)

Another important concept is the **gFEVD** of Koop, Pesaran, and Potter (1996) and Pesaran and Shin (1998). This decomposition is embedded in the `genspill()` function and captures how shocks in one variable contribute to the forecast error variance of another.

While the `genspill()` function produces several outputs (fully discussed in Section 3.2), our current interest is solely in the `gFEVD` matrix.

Each element (i, j) of the matrix quantifies the proportion of the H-step forecast error variance of variable i that can be attributed to shocks originating from variable j.
For instance, using pre-pandemic data, the gFEVD results suggest that **7.2%** of Bank of America's forecast error variance is explained by shocks from **Barclays (BCS)**, and about **6%** by shocks from **Deutsche Bank (DB)**.

```
genspill_results <- genspill(VAR_model, H = 30)
genspill_results$gFEVD
```

```
##           BAC         BK        BCS         C         CS         DB        GS
## BAC 0.9341628 0.19338713 0.07174584 0.6663017 0.11798916 0.06028896 0.5800812
## BK  0.3636745 0.84861996 0.02595194 0.3133308 0.03611692 0.01708104 0.3630035
## BCS 0.1105617 0.04713352 0.96118080 0.1137758 0.06482824 0.11523550 0.1256159
## C   0.5815652 0.18574850 0.08310222 0.9667632 0.11777901 0.05307041 0.4829849
## CS  0.1623523 0.05632249 0.05827780 0.1567158 0.95408863 0.20545501 0.1330114
## DB  0.1173690 0.04654549 0.08755388 0.1043196 0.21980032 0.93803018 0.1201977
## GS  0.5790122 0.16225563 0.08777056 0.5372057 0.08478607 0.04995554 0.9231996
## JPM 0.6204567 0.18944745 0.09349255 0.6724495 0.11208198 0.05924410 0.5367325
## MS  0.6003683 0.26112006 0.05947695 0.4635273 0.09129491 0.05815206 0.6050942
## STT 0.2704402 0.28513342 0.02768311 0.2650836 0.03344899 0.03398801 0.2973659
## WFC 0.4319118 0.16214739 0.04435500 0.4580954 0.05967095 0.03554170 0.3815070
##            JPM         MS        STT        WFC
## BAC 0.65819266 0.43345959 0.16944763 0.47917170
## BK  0.28284106 0.35771303 0.15529065 0.25641269
## BCS 0.12192095 0.09830710 0.05181303 0.06028663
## C   0.58618540 0.39354039 0.23290596 0.41561282
## CS  0.14284142 0.11647558 0.07622907 0.10152483
## DB  0.09594272 0.09836325 0.08129486 0.07489256
## GS  0.55648506 0.40694563 0.16974769 0.41660664
## JPM 0.93603194 0.37685117 0.19826444 0.55574753
## MS  0.43309480 0.84542878 0.15862593 0.34361573
## STT 0.19131216 0.33063710 0.79025181 0.16472681
## WFC 0.54340354 0.24595280 0.11973258 0.95833134
```

3.1.4 Bootstrapping

The interpretations provided above remain valid; however, bootstrapping allows us to compute **confidence intervals** around the impulse response functions, adding a layer of statistical inference.

Bootstrapping is split into four functions. First, `bootstrap_VAR()` bootstraps the VAR model itself, saving the constant, VAR coefficient matrices, and shock covariance matrix for each of `B` iterations. The first iteration is always the original (non-resampled) fitted parameters, so the original IRF can be recovered alongside the bootstrap distribution. Then, `bootstrap_gIRF()`, `bootstrap_jIRF()`, and `bootstrap_gIRF_sum()` each take the bootstrapped VAR parameters and compute confidence intervals for the generalized IRF, joint IRF, and sum of generalized IRFs, respectively — for whichever `response` variable you request.

Make sure you have sufficient memory available, as bootstrapping can be computationally intensive depending on the number of iterations (`B`). The default is `B = 10000`, and the default confidence interval is 90%, but

you can set any desired level using the `CI` argument.

In the example below, we bootstrap the VAR with 500 iterations.

```
# B is the number of bootstrap iterations (default = 10000)
# VAR_model is the object estimated using the VAR function
boot_model <- bootstrap_VAR(VAR_model, B = 500)
```

```
## Bootstrap iteration: 200
## Bootstrap iteration: 400
```

With `boot_model` computed once, we can now compute the bootstrapped confidence interval for the jIRF, the gIRF sum, or an individual gIRF, for any response variable, without re-running the bootstrap each time.

Joint IRF (jIRF) with confidence interval

```
# response is the response variable
# shocks is the set of variables where the joint shocks originate
# H is the H-step ahead
# CI sets the confidence interval level (default = 0.90)
jIRF_CI <- bootstrap_jIRF(boot_model, response = "BAC", shocks = c("BCS", "CS", "DB"),
                          H = 8, CI = 0.80)
summary(jIRF_CI)
```

```
## jIRF: The response of BAC due to joint simultaneous shocks from BCS, CS, &
##
##     CI lower bound   mean median original CI upper bound stand. dev.
## h=0         3.2425 3.6778 3.6850   3.6927         4.0933      0.3220
## h=1         2.1146 2.5650 2.5352   2.5837         3.1208      0.3935
## h=2         1.3576 1.8183 1.7893   1.8423         2.3168      0.3893
## h=3         0.8839 1.2891 1.2722   1.3154         1.7229      0.3267
## h=4         0.5922 0.9082 0.8906   0.9334         1.2467      0.2576
## h=5         0.3932 0.6366 0.6170   0.6581         0.8914      0.1985
## h=6         0.2660 0.4458 0.4292   0.4627         0.6338      0.1520
## h=7         0.1750 0.3134 0.3021   0.3259         0.4650      0.1167
## h=8         0.1174 0.2220 0.2121   0.2307         0.3400      0.0900
```

Sum of generalized IRFs (gIRF sum) with confidence interval

```
gIRF_sum_CI <- bootstrap_gIRF_sum(boot_model, response = "BAC", shocks = c("BCS", "CS", "DB"),
                                  H = 8, CI = 0.80)
summary(gIRF_sum_CI)
```

```
## gIRF sum: The sum of the generalized responses of BAC due to individual
##
##     CI lower bound   mean median original CI upper bound stand. dev.
## h=0         5.2891 6.1136 6.1440   6.1212         6.8889      0.6308
## h=1         3.4144 4.2203 4.1750   4.2448         5.2091      0.6841
## h=2         2.1661 2.9740 2.9062   3.0103         3.8405      0.6589
## h=3         1.4217 2.1025 2.0521   2.1437         2.8131      0.5490
## h=4         0.9371 1.4797 1.4516   1.5194         2.0455      0.4314
## h=5         0.6136 1.0365 1.0165   1.0708         1.4764      0.3313
## h=6         0.4153 0.7255 0.7085   0.7525         1.0609      0.2528
## h=7         0.2771 0.5096 0.4910   0.5296         0.7718      0.1932
## h=8         0.1845 0.3606 0.3459   0.3747         0.5633      0.1484
```

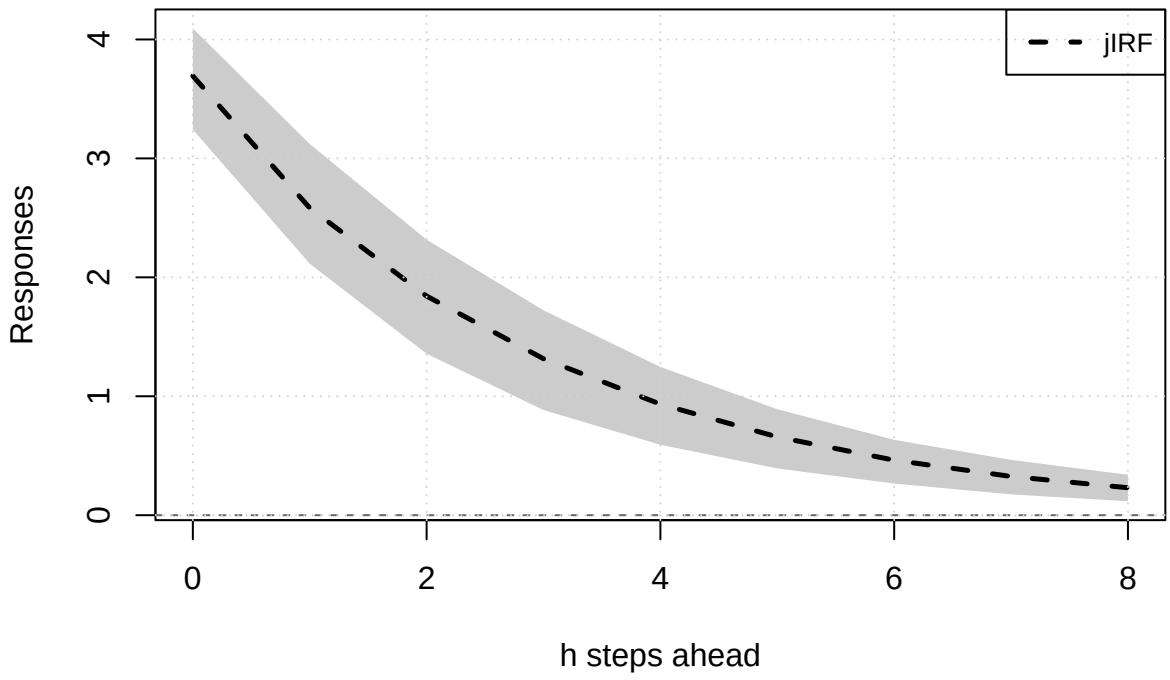
A single generalized IRF (gIRF) with confidence interval

```r
# shock is the single variable where the shock originates
gIRF_CI <- bootstrap_gIRF(boot_model, response = "BAC", shock = "BCS", H = 8, CI = 0.80)
summary(gIRF_CI)
```

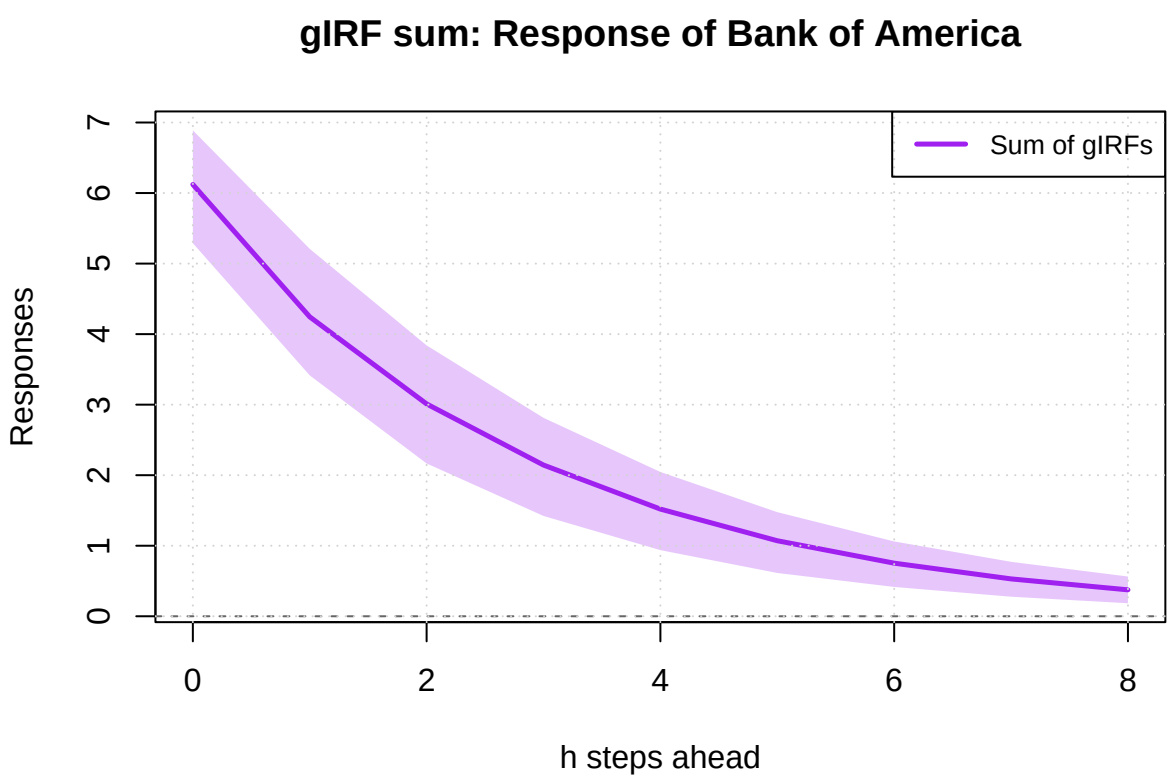
```
## gIRF: The generalized response of BAC due to a shock from BCS.
##
##     CI lower bound   mean median original CI upper bound stand. dev.
## h=0         1.4638 1.8368 1.8478   1.8387         2.1868      0.2762
## h=1         0.9421 1.3267 1.3095   1.3219         1.7406      0.3099
## h=2         0.6275 0.9887 0.9853   0.9893         1.3570      0.2940
## h=3         0.4279 0.7281 0.7256   0.7347         1.0355      0.2395
## h=4         0.3003 0.5264 0.5224   0.5359         0.7580      0.1842
## h=5         0.2090 0.3759 0.3679   0.3854         0.5444      0.1392
## h=6         0.1405 0.2675 0.2592   0.2754         0.3928      0.1053
## h=7         0.0914 0.1908 0.1838   0.1968         0.2906      0.0802
## h=8         0.0636 0.1371 0.1324   0.1413         0.2140      0.0616
```

You can visualize any of these bootstrapped IRFs directly with base R's `plot()`/`polygon()`, using the `lower`, `original`, and `upper` fields returned by each function. In all of the examples below, the shaded region shows the 80% confidence interval, and the solid/dashed line shows the original (non-bootstrapped) IRF.

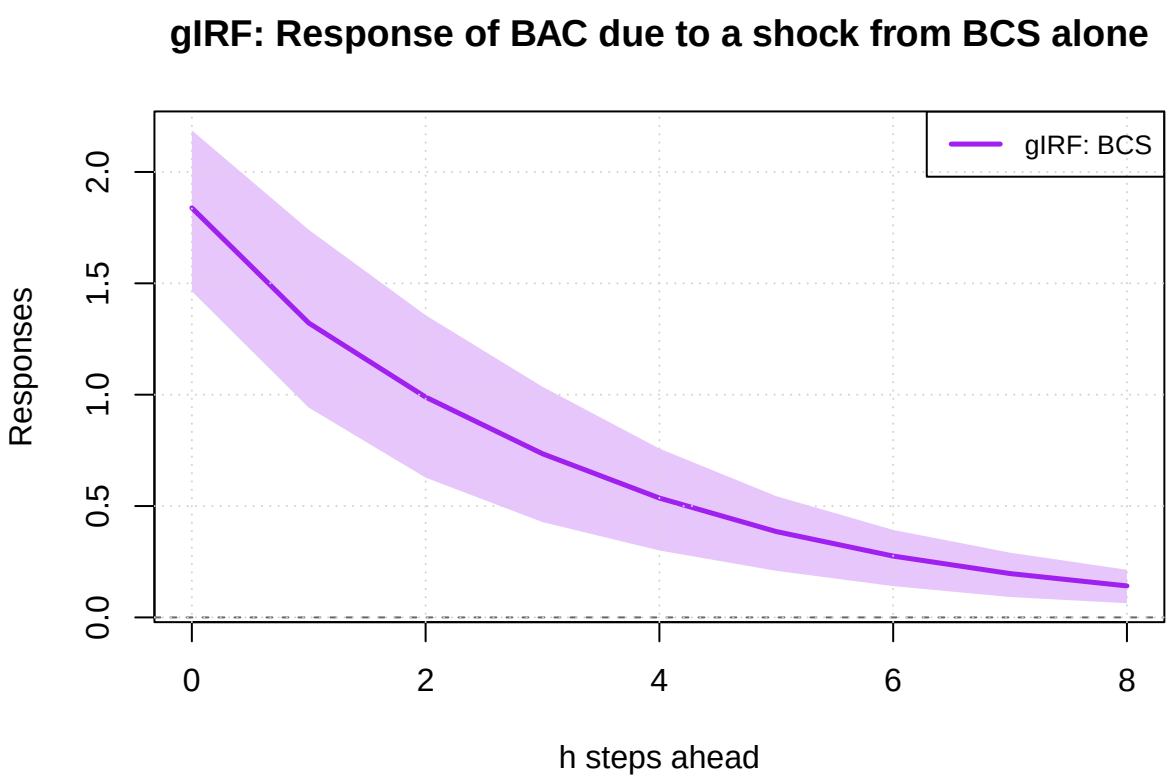
```r
horizon <- 0:jIRF_CI$H
plot(horizon, jIRF_CI$original, type = "n",
     ylim = range(c(jIRF_CI$lower, jIRF_CI$upper)),
     xlab = "h steps ahead", ylab = "Responses",
     main = "jIRF: Response of BAC due to joint shocks from BCS, CS, and DB")
polygon(c(horizon, rev(horizon)), c(jIRF_CI$lower, rev(jIRF_CI$upper)),
        col = adjustcolor("black", alpha.f = 0.2), border = NA)
lines(horizon, jIRF_CI$original, col = "black", lwd = 2.5, lty = 2)
abline(h = 0, col = "gray40", lty = 2)
grid()
legend("topright", legend = "jIRF", col = "black", lty = 2, lwd = 2.5, cex = 0.85)
```




```
horizon <- 0:gIRF_sum_CI$H
plot(horizon, gIRF_sum_CI$original, type = "n",
     ylim = range(c(gIRF_sum_CI$lower, gIRF_sum_CI$upper)),
     xlab = "h steps ahead", ylab = "Responses",
     main = "gIRF sum: Response of Bank of America")
polygon(c(horizon, rev(horizon)), c(gIRF_sum_CI$lower, rev(gIRF_sum_CI$upper)),
        col = adjustcolor("purple", alpha.f = 0.25), border = NA)
lines(horizon, gIRF_sum_CI$original, col = "purple", lwd = 2.5, lty = 1)
abline(h = 0, col = "gray40", lty = 2)
grid()
legend("topright", legend = "Sum of gIRFs", col = "purple", lty = 1, lwd = 2.5, cex = 0.85)
```




```
horizon <- 0:gIRF_CI$H
plot(horizon, gIRF_CI$original, type = "n",
     ylim = range(c(gIRF_CI$lower, gIRF_CI$upper)),
     xlab = "h steps ahead", ylab = "Responses",
     main = "gIRF: Response of BAC due to a shock from BCS alone")
polygon(c(horizon, rev(horizon)), c(gIRF_CI$lower, rev(gIRF_CI$upper)),
        col = adjustcolor("purple", alpha.f = 0.25), border = NA)
lines(horizon, gIRF_CI$original, col = "purple", lwd = 2.5, lty = 1)
abline(h = 0, col = "gray40", lty = 2)
grid()
legend("topright", legend = "gIRF: BCS", col = "purple", lty = 1, lwd = 2.5, cex = 0.85)
```



3.1.5 Replicating Figure 3 of Wiesen and Beaumont (2024)

We can replicate the pre-pandemic column of Figure 3 of Wiesen and Beaumont (2024), which shows the jIRF (with confidence interval), the three individual gIRFs, and the sum of gIRFs, due to joint shocks from Barclays (BCS), Credit Suisse (CS), and Deutsche Bank (DB), for a given response variable.

```
plot_fig3_panel <- function(boot_model, response, main) {
  shocks <- c("BCS", "CS", "DB")
  H <- 8
  CI <- 0.80
  horizon <- 0:H

  jIRF_CI <- bootstrap_jIRF(boot_model, response = response, shocks = shocks,
                            H = H, CI = CI)
  gIRF_sum_CI <- bootstrap_gIRF_sum(boot_model, response = response, shocks = shocks,
                                    H = H, CI = CI)
  gIRF_list <- lapply(shocks, function(s) {
    bootstrap_gIRF(boot_model, response = response, shock = s, H = H, CI = CI)
  })

  g_colors <- c("blue", "red", "darkgreen")
  g_ltys <- c(3, 2, 4)

  y_range <- range(c(jIRF_CI$lower, jIRF_CI$upper,
                     gIRF_sum_CI$lower, gIRF_sum_CI$upper,
                     sapply(gIRF_list, function(g) range(g$lower, g$upper))))
```

```
  plot(horizon, jIRF_CI$original, type = "n", xlim = c(0, H), ylim = y_range,
       xlab = "h steps ahead", ylab = "impulse response", main = main,
       cex.main = 1.1, cex.lab = 0.95, cex.axis = 0.9)

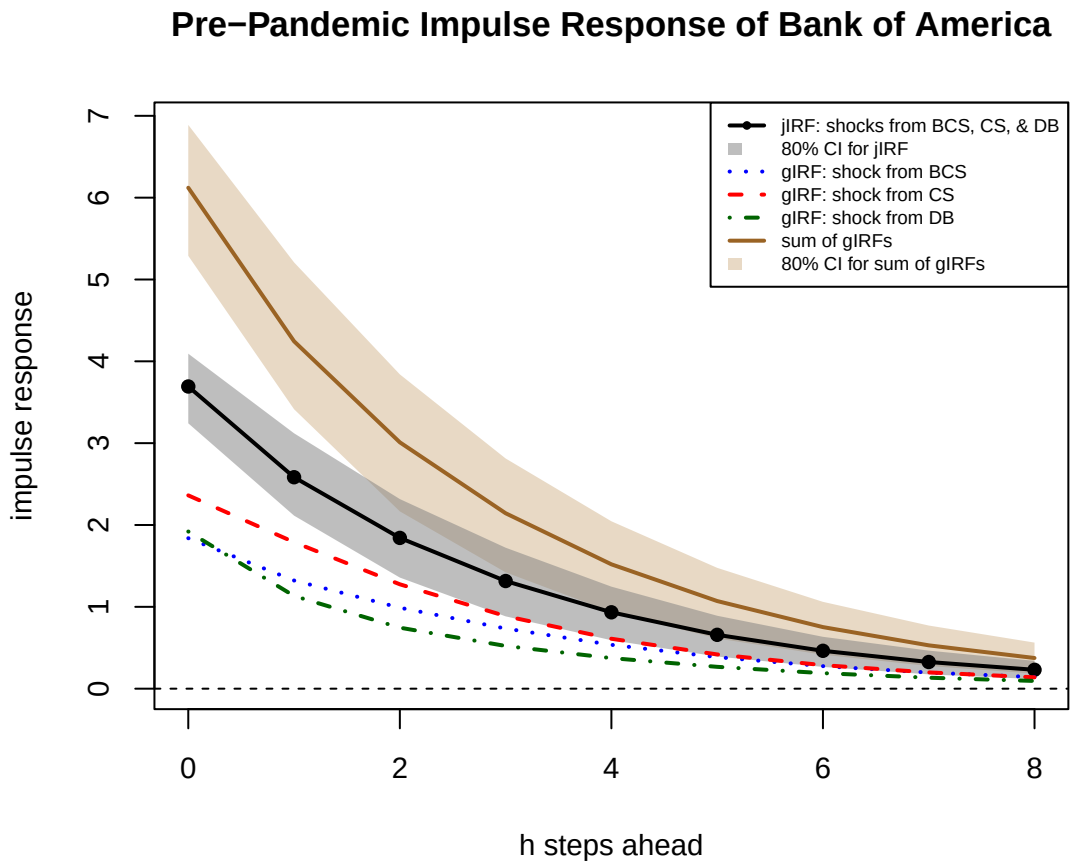
  polygon(c(horizon, rev(horizon)), c(gIRF_sum_CI$lower, rev(gIRF_sum_CI$upper)),
          col = adjustcolor("tan", alpha.f = 0.5), border = NA)
  polygon(c(horizon, rev(horizon)), c(jIRF_CI$lower, rev(jIRF_CI$upper)),
          col = adjustcolor("gray50", alpha.f = 0.5), border = NA)

  lines(horizon, gIRF_sum_CI$original, col = "#9A6324", lwd = 2)
  lines(horizon, jIRF_CI$original, col = "black", lwd = 2, type = "o", pch = 16)
  for (i in seq_along(shocks)) {
    lines(horizon, gIRF_list[[i]]$original, col = g_colors[i], lwd = 2, lty = g_ltys[i])
  }
  abline(h = 0, col = "black", lty = 2)

  legend("topright", cex = 0.6,
         legend = c("jIRF: shocks from BCS, CS, & DB", "80% CI for jIRF",
                    "gIRF: shock from BCS", "gIRF: shock from CS", "gIRF: shock from DB",
                    "sum of gIRFs", "80% CI for sum of gIRFs"),
         col = c("black", "gray50", g_colors, "#9A6324", "tan"),
         lty = c(1, NA, g_ltys, 1, NA),
         lwd = c(2, NA, 2, 2, 2, 2, NA),
         pch = c(16, NA, NA, NA, NA, NA, NA),
         fill = c(NA, adjustcolor("gray50", alpha.f = 0.5), NA, NA, NA, NA,
                  adjustcolor("tan", alpha.f = 0.5)),
         border = NA, merge = TRUE, seg.len = 2)
}
```

Now produce the first panel of Figure 3's pre-pandemic column, the pre-pandemic impulse response of Bank of America:

```
plot_fig3_panel(boot_model, "BAC", "Pre-Pandemic Impulse Response of Bank of America")
```



The remaining panels of Figure 3 (Citigroup and JP Morgan Chase) can be reproduced the same way, by calling `plot_fig3_panel()` with `response = "C"` or `response = "JPM"`.

3.2 Lastrapes and Wiesen (2021) and Lastrapes and Wiesen (2025) - Joint Spillover Metrics

We refer users to the works of Lastrapes and Wiesen (2021) and Lastrapes and Wiesen (2025) to understand the concept of **joint spillover metrics**, and to Diebold and Yilmaz (2012) and Diebold and Yilmaz (2014) for the **generalized variant**.

As with the joint impulse response function and the joint forecast error variance decomposition (from section 3.1), you must estimate a K-variable VAR model first before computing the (non rolling window) spillover metrics. As illustrated previously, say you want to load the pre-pandemic bank volatility data from Wiesen and Beaumont (2024). For more information about this dataset, see section 3.1.

```
#Load and prepare the data as follows:
data(Volatility_data_pre)

Data <- Volatility_data_pre
date <- attr(Volatility_data_pre, "date")
```

```
#------------- Run a VAR model------
# Estimate VAR with 1 lag, as used in the paper
VAR_model <- VAR(Data, p = 1, type = "const")
```

Once the VAR_model object above is available, the `jointspill()` function can be used to compute the joint spillover metrics.

The `jointspill()` function produces the following results:

1. The K x K **gFEVD matrix** (`gFEVD`)
2. The **joint total spillover from all others to variable *i*** (`jnt_from_others_to_i`)
3. The **joint total spillover from variable *i* to all others** (`jnt_to_others_from_i`)
4. The **joint net total spillover** (`jnt_net`)
5. The **joint spillover index** (`jSOI`)
6. The **lambda scaling factor** (`lambda`)
7. The **scaled gFEVD matrix** (`gFEVD_tilde`)

As before, `H` denotes the number of forecast periods ahead (i.e., the forecast horizon).

```
jointspill_results <- jointspill(VAR_model, H = 30)
```

If users want to extract any of the seven outputs calculated by `jointspill()`, they can use the `$` extraction operator. For example, `jointspill_results$gFEVD` returns the K x K generalized forecast error variance decomposition (gFEVD) matrix.

Additionally, users can easily view outputs (1) through (5) using the generic `summary()` function. This summary presents a well-structured table:

- The upper-left K x K portion contains the gFEVD matrix.
- The last column shows the **joint total spillover from all others** to each variable (*jnt_from_others_to_i*).
- The second-to-last row displays the **joint total spillover from each variable to all others** (*jnt_to_others_from_i*).
- The final row contains the **joint net total spillovers** (*jnt_net*).
- The **joint spillover index (jSOI)** is reported in the bottom-right corner of the table.

This summary can be generated using:

```
summary(jointspill_results)
```

```
## BAC BK BCS C CS DB GS JPM MS
## BAC 93.42 19.34 7.17 66.63 11.80 6.03 58.01 65.82 43.35
## BK 36.37 84.86 2.60 31.33 3.61 1.71 36.30 28.28 35.77
## BCS 11.06 4.71 96.12 11.38 6.48 11.52 12.56 12.19 9.83
## C 58.16 18.57 8.31 96.68 11.78 5.31 48.30 58.62 39.35
## CS 16.24 5.63 5.83 15.67 95.41 20.55 13.30 14.28 11.65
## DB 11.74 4.65 8.76 10.43 21.98 93.80 12.02 9.59 9.84
## GS 57.90 16.23 8.78 53.72 8.48 5.00 92.32 55.65 40.69
## JPM 62.05 18.94 9.35 67.24 11.21 5.92 53.67 93.60 37.69
## MS 60.04 26.11 5.95 46.35 9.13 5.82 60.51 43.31 84.54
## STT 27.04 28.51 2.77 26.51 3.34 3.40 29.74 19.13 33.06
## WFC 43.19 16.21 4.44 45.81 5.97 3.55 38.15 54.34 24.60
## To others from i 95.07 39.37 15.84 92.92 23.23 17.04 89.82 89.48 70.81
## NET 12.43 -13.67 -6.72 19.65 -9.53 -16.36 17.68 6.64 -4.48
##                      STT    WFC From others to i
## BAC                16.94  47.92            82.64
## BK                 15.53  25.64            53.04
## BCS                 5.18   6.03            22.56
## C                  23.29  41.56            73.27
## CS                  7.62  10.15            32.76
## DB                  8.13   7.49            33.40
## GS                 16.97  41.66            72.13
## JPM                19.83  55.57            82.84
```

```
## MS                 15.86  34.36           75.28
## STT                79.03  16.47           50.75
## WFC                11.97  95.83           60.97
## To others from i   35.01  71.06            jSOI
## NET               -15.74  10.09           58.15
```

As an example to interpret the results, the value in the first row and second column (the 1,2 element of the gFEVD matrix) implies that 19.34% of Bank of America's (BAC) forecast error variance can be explained by the shocks of Bank of New York Mellon (BK). The first value in the "FROM Others" column (the first element of **jnt_from_others_to_i**) implies that 82.64% of Bank of America's forecast error variance can be jointly explained by the shocks of the other ten banks in the model. The first value in the "TO Others" row (the first element of **jnt_to_others_from_i**) implies that the joint total spillover from Bank of America to the other ten banks is 95.07%. Note that this value can exceed 100%. The first value in the "Net" row (the first element of **jnt_net**) implies that the joint net total spillover for Bank of America is 12.43%. Because this value is positive, Bank of America transmits more shocks to others banks than it receives. Lastly, the value in the bottom-right is the joint spillover index (**jSOI**). This indicates that, on average, 58.15% of a bank's forecast error variance can be explained by jointly conditioning on the shocks of the other ten banks in the model.

Similar to the joint spillover results, the generalized spillover metrics of Diebold and Yilmaz (2012) and Diebold and Yılmaz (2014) can also be estimated using the `genspill()` function.

This function returns the following outputs:

1. The K x K generalized spillover table (**gSOT**)
2. The generalized total spillover **from all others** to variable i (**gen_from_others_to_i**)
3. The generalized total spillover **from variable *i*** to all others (**gen_to_others_from_i**)
4. The **generalized net** total spillover (**gen_net**)
5. The **generalized spillover index** (**gSOI**)

You can compute these metrics by running:

```
#Compute the generalized spillover results using the estimated VAR model
#with a time horizon of H=30.
genspill_results <- genspill(VAR_model, H = 30)
```

As in Diebold and Yilmaz (2012) and Diebold and Yılmaz (2014), users can use the generic `summary()` function to generate a summarized version of the generalized spillover results.

- The **upper-left K x K** part of the table represents the **gSOT matrix**.
- The **last column** shows the generalized total spillover **from all others** to each variable.
- The **second-to-last row** displays the generalized total spillover **to all others** from each variable.
- The **last row** contains the generalized **net** total spillover.
- The **bottom-right** cell presents the overall **generalized spillover index (gSOI)**.

```
summary(genspill_results)
```

```
## BAC BK BCS C CS DB GS JPM MS
## BAC 21.40 4.43 1.64 15.27 2.70 1.38 13.29 15.08 9.93
## BK 12.04 28.10 0.86 10.38 1.20 0.57 12.02 9.37 11.84
## BCS 5.91 2.52 51.38 6.08 3.47 6.16 6.72 6.52 5.26
## C 14.19 4.53 2.03 23.58 2.87 1.29 11.78 14.30 9.60
## CS 7.50 2.60 2.69 7.24 44.10 9.50 6.15 6.60 5.38
## DB 5.91 2.35 4.41 5.26 11.08 47.27 6.06 4.84 4.96
## GS 14.57 4.08 2.21 13.52 2.13 1.26 23.23 14.00 10.24
## JPM 14.26 4.35 2.15 15.46 2.58 1.36 12.34 21.51 8.66
## MS 15.32 6.66 1.52 11.83 2.33 1.48 15.44 11.05 21.57
## STT 10.05 10.60 1.03 9.85 1.24 1.26 11.05 7.11 12.29
```

```
## WFC 12.55 4.71 1.29 13.31 1.73 1.03 11.09 15.79 7.15
## To others from i 112.31 46.84 19.83 108.19 31.33 25.30 105.93 104.66 85.31
## NET 33.72 -25.06 -28.79 31.78 -24.56 -27.43 29.16 26.17 6.88
##                      STT    WFC From others to i
## BAC                 3.88  10.98            78.60
## BK                  5.14   8.49            71.90
## BCS                 2.77   3.22            48.62
## C                   5.68  10.14            76.42
## CS                  3.52   4.69            55.90
## DB                  4.10   3.77            52.73
## GS                  4.27  10.48            76.77
## JPM                 4.56  12.77            78.49
## MS                  4.05   8.77            78.43
## STT                29.38   6.12            70.62
## WFC                 3.48  27.85            72.15
## To others from i   41.45  79.45             gSOI
## NET               -29.17   7.30            69.15
```

As an example to interpret the results, the value in the first row and second column (the 1,2 element of the gSOT matrix) shows the generalized bilateral spillover from Bank of New York Mellon (BK) to Bank of America (BAC) is 4.43%. Because of the row-sum normalization used by DY (Diebold and Yilmaz 2012; Diebold and Yılmaz 2014), this 4.43% is NOT the same thing as the percent of Bank of America's forecast error variance explained by the shocks of Bank of New York Mellon. The first value in the "FROM Others" column (the first element of gen_from_others_to_i) shows the generalized total spillover from the other ten banks to Bank of America is 78.60%. Because of the row-sum normalization and single-variable conditioning sets used by DY (Diebold and Yilmaz 2012; Diebold and Yılmaz 2014), this 78.60% is NOT the same thing as the percent of Bank of America's forecast error variance jointly explained by the shocks of the other ten banks in the model. The first value in the "TO Others" row (the first element of gen_to_others_from_i) shows the generalized total spillover from Bank of America to the other ten banks is 112.31%. The first value in the "Net" row (the first element of gen_net) implies that the generalized net total spillover for Bank of America is 33.72%. Lastly, the value in the bottom-right is the generalized spillover index (gSOI) of 69.15%. Because of the row-sum normalization and single-variable conditioning sets used by DY (Diebold and Yilmaz 2012; Diebold and Yılmaz 2014), this 69.15% is not the same thing as the average percent of a bank's forecast error variance jointly explained by the shocks of the other ten banks in the model.

3.2.1 Rolling Window Analyses

To perform rolling window analyses for either the **joint spillover metrics** or its **generalized variant**, users must first estimate a rolling window VAR model. The parameter `win` specifies the window size. For example, the following line of code will estimate a rolling window VAR model of one lag and a window length of 150 periods using the pre-pandemic bank volatility data from previous sections

```
# p is the lag
# win is the window size
# const is TRUE if you want to include a constant, or FALSE if not.
# dates is the date vector extracted earlier (attr(Volatility_data_pre, "date"));
# VAR_roll() uses it (together with win) to compute the end-of-window date for each
# rolling window, which is carried through as end_of_window_date in its output.
rolling_model <- VAR_roll(data = Data, p = 1, win = 150, const = TRUE, dates = date)
```

```
## rolling window progress (%): 8.01
## rolling window progress (%): 16.03
## rolling window progress (%): 24.04
## rolling window progress (%): 32.05
## rolling window progress (%): 40.06
## rolling window progress (%): 48.08
```

```
## rolling window progress (%): 56.09
## rolling window progress (%): 64.1
## rolling window progress (%): 72.12
## rolling window progress (%): 80.13
## rolling window progress (%): 88.14
## rolling window progress (%): 96.15
## rolling window progress (%): 100
```

If users want to extract any of the four outputs produced by the `VAR_roll()` function, they can do so using the `$` extraction operator. The available components are:

1. `VARcoeff` - VAR coefficient matrices for each rolling window
2. `constant` - Intercepts for each rolling window
3. `Sigma` - Residual (shock) covariance matrices for each rolling window
4. `end_of_window_date` - The end-of-window date for each rolling window, computed from the `dates` and `win` inputs

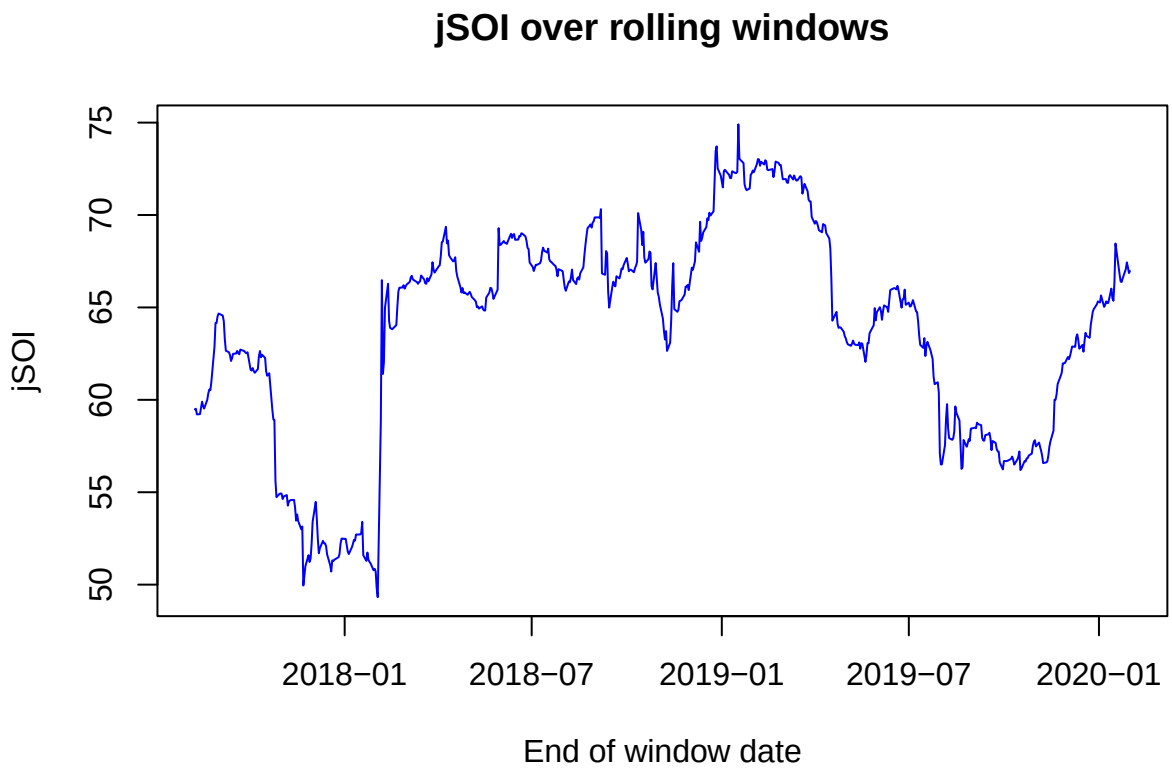
For example, `rolling_model$VARcoeff` will return the rolling VAR coefficients.

Compute the joint spillover results using the rolling window VAR model with a forecast horizon `H`. The `dates` input to `VAR_roll()` should be the date vector extracted earlier (`date <- attr(Volatility_data_pre, "date")`). The "date" input should not be truncated or shortened. The R code uses the `win` input together with `dates` to calculate the end-of-window date for each rolling window.

```r
# H is the horizon.
# jointspill_roll() carries the end-of-window dates through from rolling_model
# (computed by VAR_roll() using the win input), so no separate dates argument is needed.
joint_spill_results_roll <- jointspill_roll(rolling_model, H = 30)
```

As an example, if users wish to plot the jSOI over rolling windows, they can run the following.

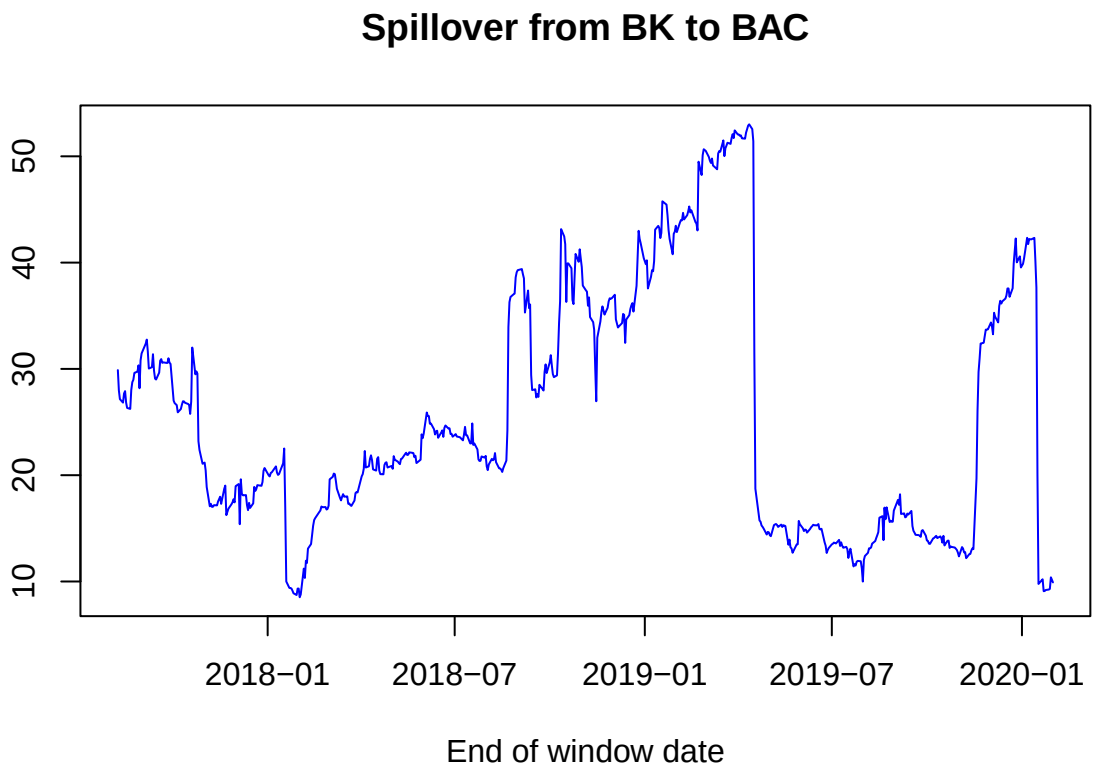
```r
#Plot the joint spillover index over rolling windows.
plot(joint_spill_results_roll$end_of_window_date, joint_spill_results_roll$jSOI,
     type = "l", col = "blue", lwd = 1,
     ylim = range(joint_spill_results_roll$jSOI),
     xlab = "End of window date", ylab = "jSOI",
     main = "jSOI over rolling windows")
```



If users wish to plot the 1,2 element of the gFEVD matrix (the percent of Bank of America's forecast error variance explained by the shocks of Bank of New York Mellon) over rolling windows, they can run the following

```
#Plot the 1,2 element of the gFEVD matrix over the rolling windows.
#This is the fraction of the forecast error variance of BAC
#that can be explained by the shocks of BK over rolling windows.

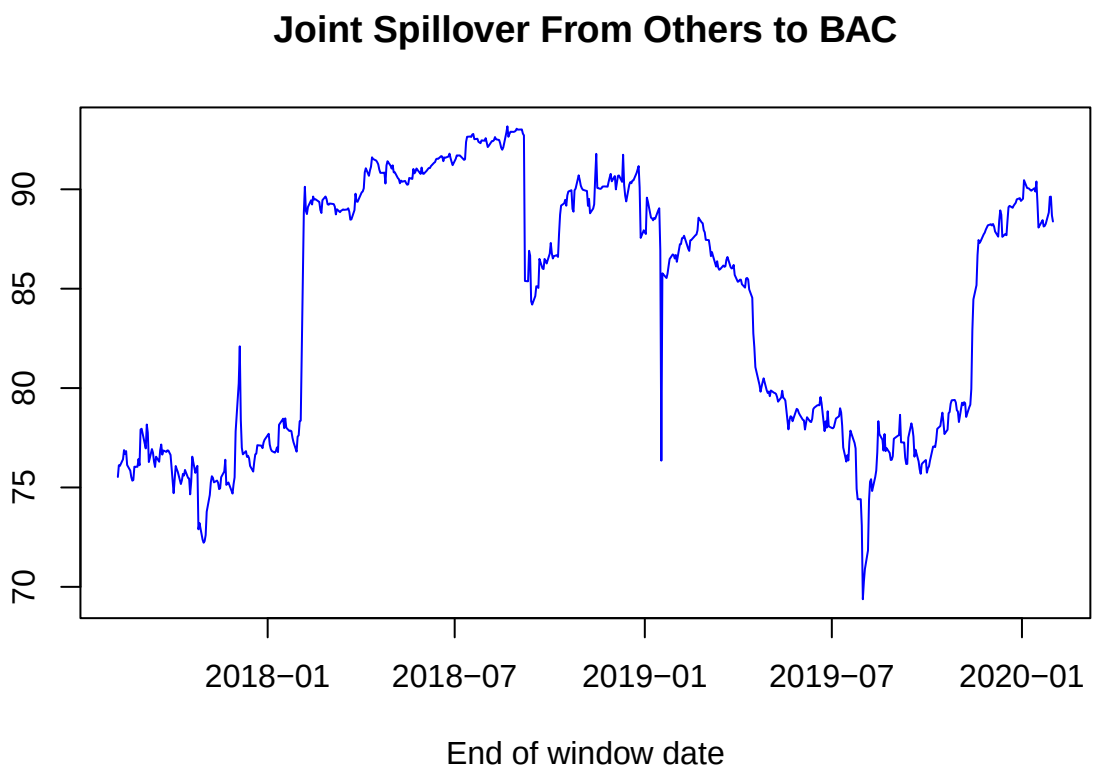
plot(joint_spill_results_roll$end_of_window_date,
     joint_spill_results_roll$gFEVD["BAC","BK",], type = "l", col = "blue", lwd = 1,
     ylim = range(joint_spill_results_roll$gFEVD["BAC","BK",]),
     xlab = "End of window date", ylab = "",
     main = "Spillover from BK to BAC")
```



If users wish to plot the joint total spillover from all other banks to Bank of America over rolling windows, they can run the following.

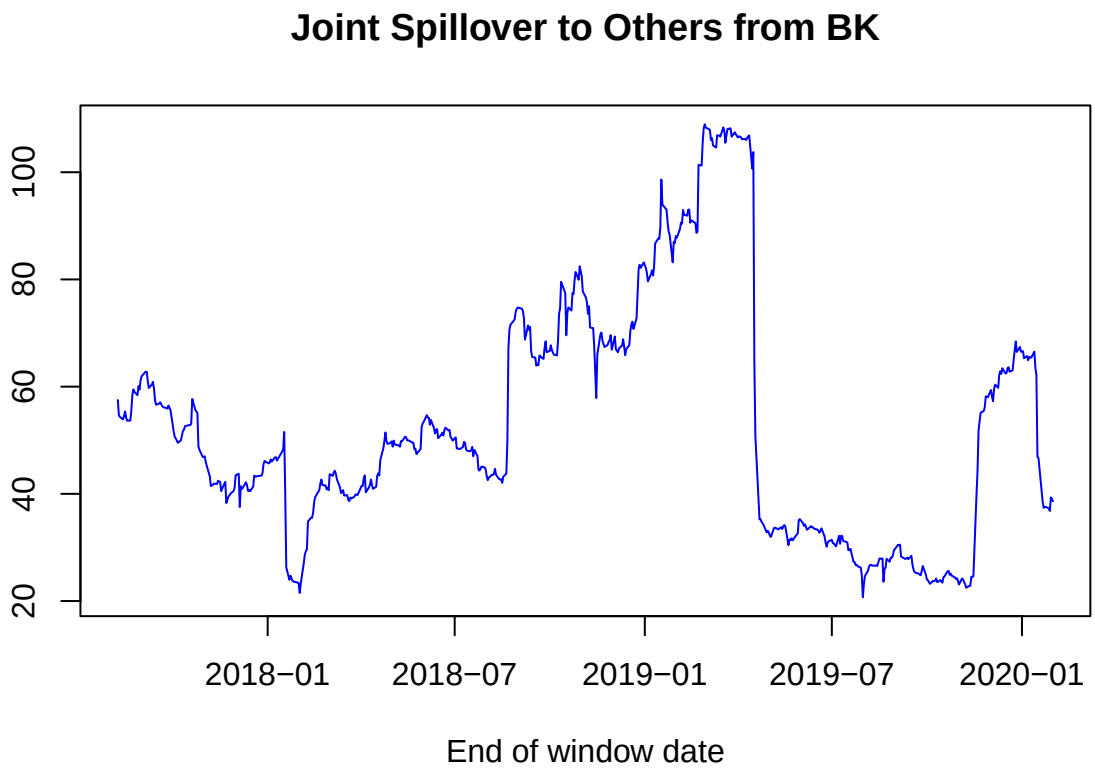
```
##Plot the joint total spillover from all others to BAC over
#rolling windows.

plot(joint_spill_results_roll$end_of_window_date,
     joint_spill_results_roll$jnt_from_others_to_i["BAC",],
     type = "l", col = "blue", lwd = 1,
     ylim = range(joint_spill_results_roll$jnt_from_others_to_i["BAC",]),
     xlab = "End of window date", ylab = "",
     main = "Joint Spillover From Others to BAC")
```



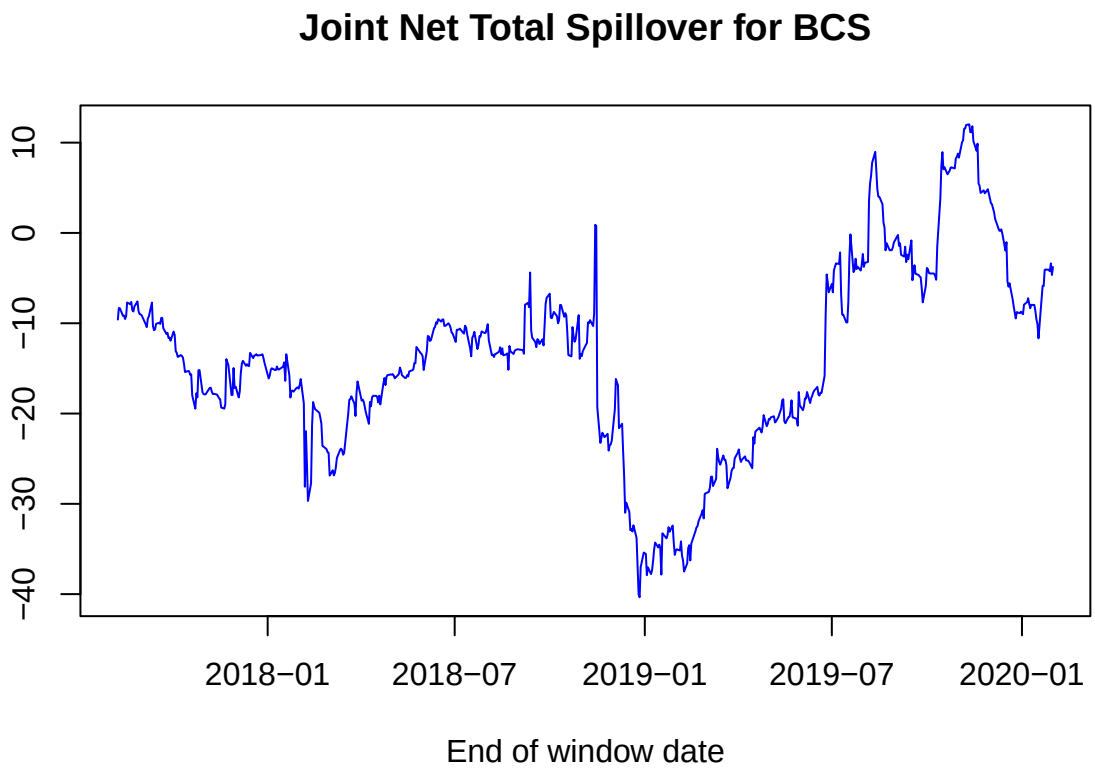
If users wish to plot the joint total spillover from Bank of New York Mellon to the other banks over rolling windows, they can run the following.

```
###Plot the joint total spillover to all others from BK over
#rolling windows.
plot(joint_spill_results_roll$end_of_window_date,
     joint_spill_results_roll$jnt_to_others_from_i["BK",],
     type = "l", col = "blue", lwd = 1,
     ylim = range(joint_spill_results_roll$jnt_to_others_from_i["BK",]),
     xlab = "End of window date", ylab = "",
     main = "Joint Spillover to Others from BK")
```



If users wish to plot the joint net total spillover for Barclays over rolling windows, they can run the following.

```
#Plot the joint net total spillover for BCS over rolling windows.
plot(joint_spill_results_roll$end_of_window_date,
     joint_spill_results_roll$jnt_net["BCS",], type = "l", col = "blue", lwd = 1,
     ylim = range(joint_spill_results_roll$jnt_net["BCS",]),
     xlab = "End of window date", ylab = "",
     main = "Joint Net Total Spillover for BCS")
```



The command `genspill_roll()` can be used to compute the generalized spillover metrics over rolling windows. Notice that the arguments of the `genspill_roll()` command are:

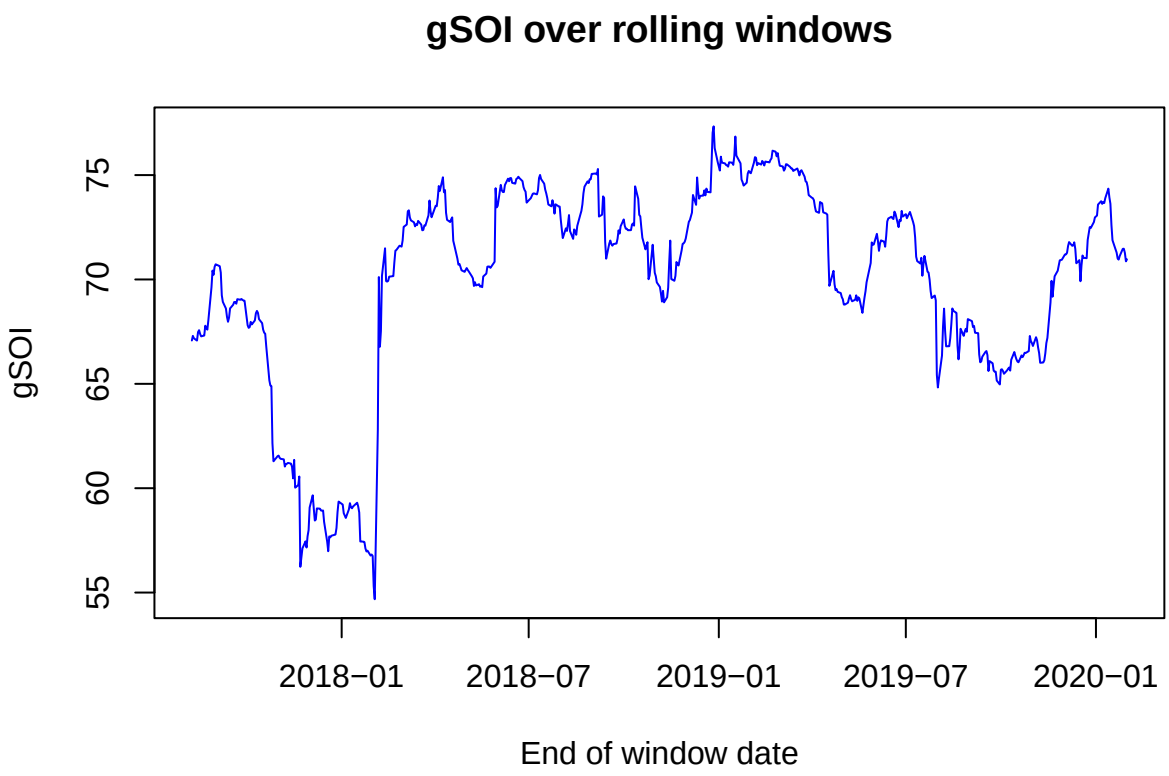
1. `rolling_model` - the rolling window VAR object
2. `H` - the forecast error horizon

Like `jointspill_roll()`, the end-of-window dates are carried through from `rolling_model`, so there is no separate `dates` argument.

```
#--------------- Diebold-Yilmaz Spillover rolling window-------
##Compute the generalized spillover results using the rolling window VAR
#model with a time horizon of H=30.
gen_spill_result_roll <- genspill_roll(rolling_model, H = 30)
```

As an example, if users wish to plot the gSOI over rolling windows, they can run the following

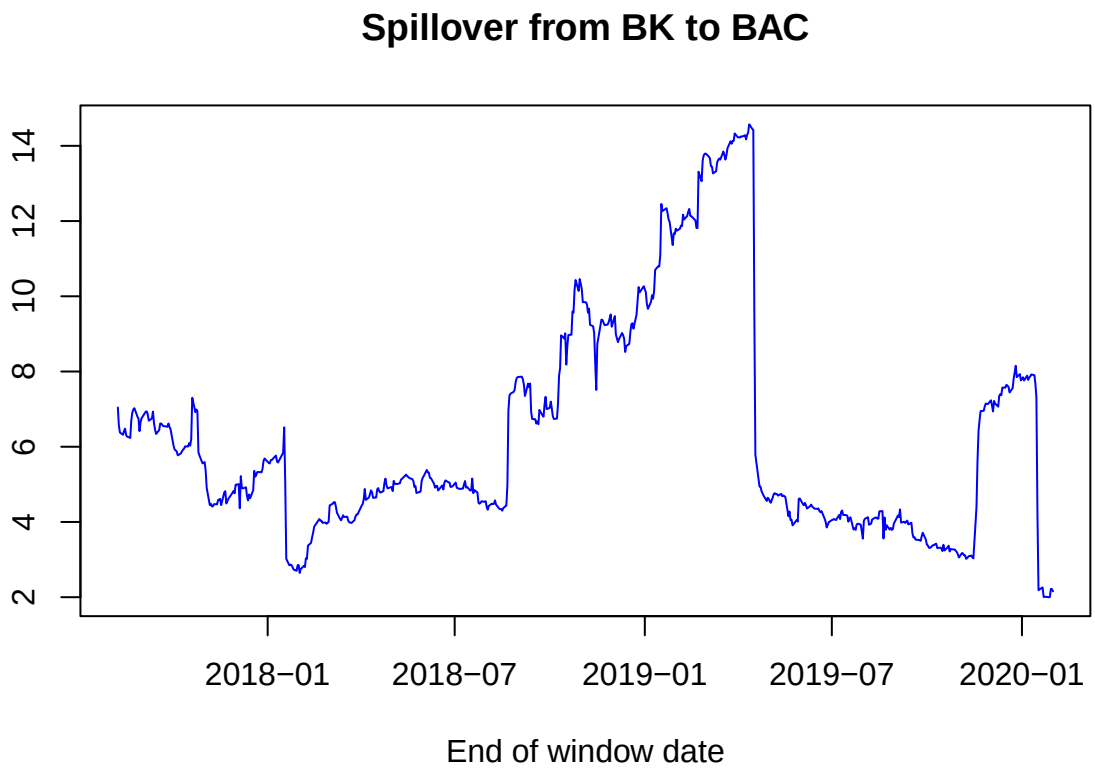
```
#Plot the generalized spillover index over rolling windows.
plot(gen_spill_result_roll$end_of_window_date, gen_spill_result_roll$gSOI,
     type = "l", col = "blue", lwd = 1,
     ylim = range(gen_spill_result_roll$gSOI),
     xlab = "End of window date", ylab = "gSOI",
     main = "gSOI over rolling windows")
```



If users wish to plot the 1,2 element of the gSOT matrix (generalized bilateral spillover to Bank of America from Bank of New York Mellon) over rolling windows, they can run the following

```
#Plot the 1,2 element of the gSOT matrix over the rolling windows.
#This is the generalized bilateral spillover from BK to BAC over rolling windows.

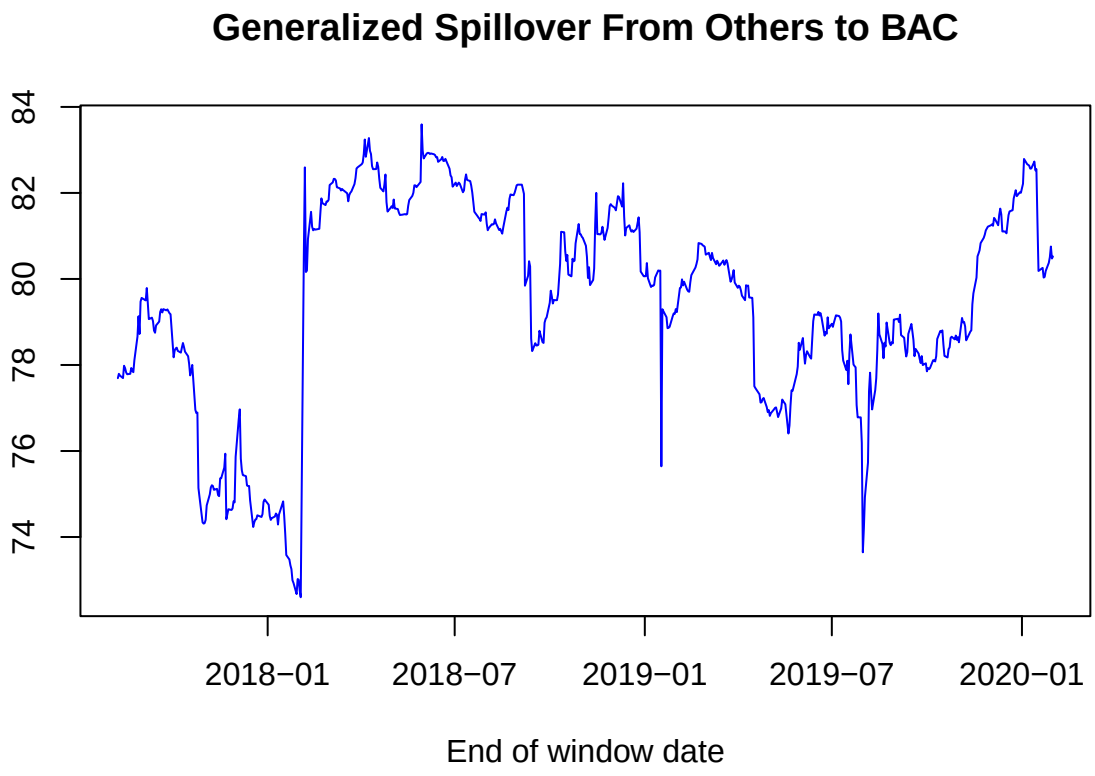
plot(gen_spill_result_roll$end_of_window_date,
     gen_spill_result_roll$gSOT["BAC","BK",],
     type = "l", col = "blue", lwd = 1,
     ylim = range(gen_spill_result_roll$gSOT["BAC","BK",]),
     xlab = "End of window date", ylab = "",
     main = "Spillover from BK to BAC")
```



If users wish to plot the generalized total spillover from all other banks to Bank of America over rolling windows, they can run the following.

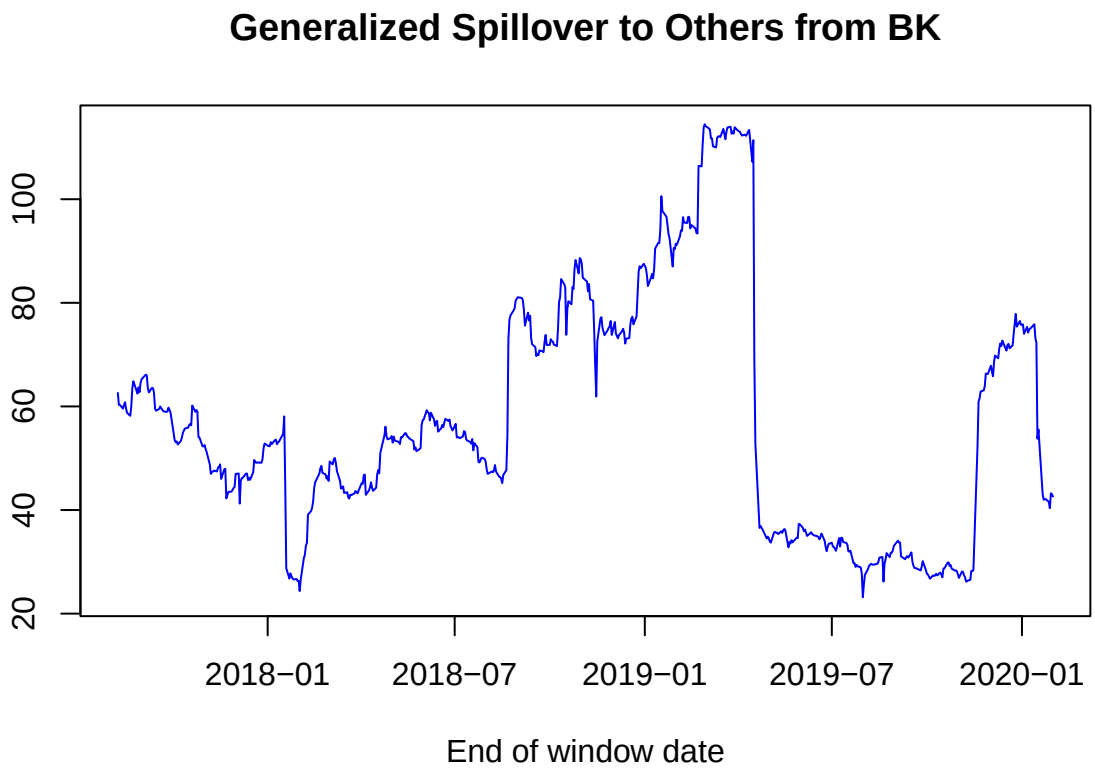
```
##Plot the generalized total spillover from all others to BAC over
#rolling windows.

plot(gen_spill_result_roll$end_of_window_date,
     gen_spill_result_roll$gen_from_others_to_i["BAC",],
     type = "l", col = "blue", lwd = 1,
     ylim = range(gen_spill_result_roll$gen_from_others_to_i["BAC",]),
     xlab = "End of window date", ylab = "",
     main = "Generalized Spillover From Others to BAC")
```



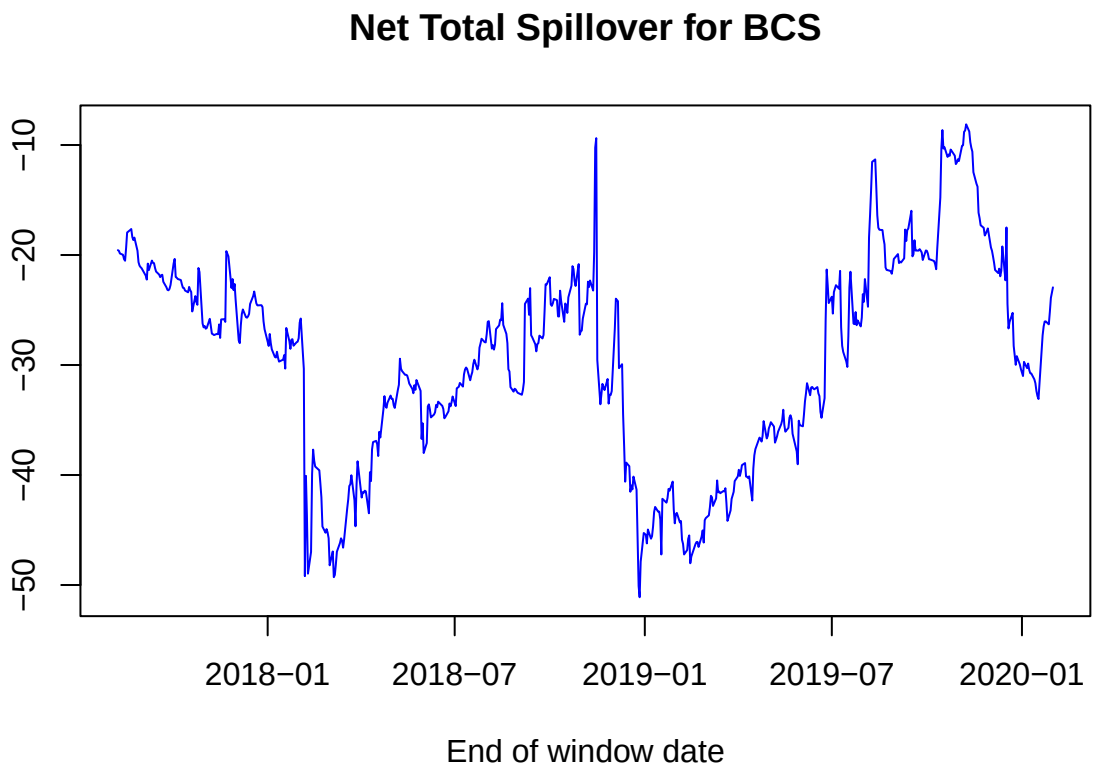
If users wish to plot the generalized total spillover from Bank of New York Mellon to the other banks over rolling windows, they can run the following

```
###Plot the generalized total spillover to all others from BK over
#rolling windows.
plot(gen_spill_result_roll$end_of_window_date,
     gen_spill_result_roll$gen_to_others_from_i["BK",],
     type = "l", col = "blue", lwd = 1,
     ylim = range(gen_spill_result_roll$gen_to_others_from_i["BK",]),
     xlab = "End of window date", ylab = "",
     main = "Generalized Spillover to Others from BK")
```



If users wish to plot the generalized net total spillover for Barclays over rolling windows, they can run the following.

```
#Plot the generalized net total spillover for BCS over rolling windows.
plot(gen_spill_result_roll$end_of_window_date,
     gen_spill_result_roll$gen_net["BCS",],
     type = "l", col = "blue", lwd = 1,
     ylim = range(gen_spill_result_roll$gen_net["BCS",]),
     xlab = "End of window date", ylab = "",
     main = "Net Total Spillover for BCS")
```



#References

Diebold, Francis X, and Kamil Yilmaz. 2012. "Better to Give Than to Receive: Predictive Directional Measurement of Volatility Spillovers." *International Journal of Forecasting* 28 (1): 57–66.

Diebold, Francis X, and Kamil Yılmaz. 2014. "On the Network Topology of Variance Decompositions: Measuring the Connectedness of Financial Firms." *Journal of Econometrics* 182 (1): 119–34.

Koop, Gary, M Hashem Pesaran, and Simon M Potter. 1996. "Impulse Response Analysis in Nonlinear Multivariate Models." *Journal of Econometrics* 74 (1): 119–47.

Lastrapes, William D, and Thomas FP Wiesen. 2021. "The Joint Spillover Index." *Economic Modelling* 94: 681–91.

———. 2025. "Regional Bank Failures and Volatility Transmission." *Journal of Financial Stability* 78: 101404.

Pesaran, H Hashem, and Yongcheol Shin. 1998. "Generalized Impulse Response Analysis in Linear Multivariate Models." *Economics Letters* 58 (1): 17–29.

Wiesen, Thomas FP, and Paul M Beaumont. 2024. "A Joint Impulse Response Function for Vector Autoregressive Models." *Empirical Economics* 66 (4): 1553–85.